

Stratégie de test : Concevez une stratégie de test :

Scénarios prévus :

Exigence 1 : Gestion des comptes utilisateurs :

Description : Permet la création, modification, suppression, et gestion des informations des comptes utilisateurs.

Scénario 1 : Création d'un compte utilisateur :

Objectif : Vérifier que l'utilisateur peut créer un compte avec les informations requises et que les contrôles sont bien appliqués.

Exigences associées : TOM-1.1, TOM-1.4, TOM-6.1, TOM-6.2, TOM-6.3, TOM-6.4, TOM-6.5, TOM-6.6, TOM-6.7

Scénario 2 : Évolution automatique de la date limite de naissance

Objectif : S'assurer que la règle d'âge minimum à l'inscription est appliquée correctement et évolue automatiquement en fonction de l'année en cours.

Exigences associées : TOM-6-3

Scénario 3 : modification des informations des comptes :

Objectif : Vérifier si un utilisateur peut modifier certaines informations après la création de son compte.

Exigences associées : TOM-1.2

Scénario 4 : Suppression d'un compte

Objectif : Vérifier que la suppression d'un compte est bien prise en charge et qu'elle suit les règles définies.

Exigences associées : TOM-1.3

Scénario 5 : Téléversement d'un justificatif :

Objectif : Vérifier que l'utilisateur peut téléverser un justificatif dans les formats et limites autorisés.

Exigences associées : TOM-7.1, TOM-7.2, TOM-7.4

Scénario 6 : Vérification du consentement et du stockage des données :

Objectif : Vérifier que les données personnelles sont stockées avec le consentement explicite de l'utilisateur et qu'elles ne sont pas conservées inutilement.

Exigences associées : TOM-1.5

Scénario 7 : Suppression automatique des données après inactivité :

Objectif : Vérifier que les données utilisateur sont supprimées ou anonymisées après la durée légale de conservation

Exigences associées : TOM-1.5

Exigence 2 : Authentification et sécurité :

Description : Garantir que l'utilisateur puisse s'authentifier en toute sécurité, y compris l'authentification biométrique et à double facteur.

Scénario 8 : Authentification avec double facteur :

Objectif : Vérifier que l'authentification à double facteur fonctionne correctement et que les erreurs sont bien gérées.

Exigences associées : TOM-8.1, TOM-8.3, TOM-8.4

Scénario 9 : Activation de l'authentification biométrique :

Objectif : Vérifier que l'utilisateur peut activer l'authentification biométrique et l'utiliser pour se connecter.

Exigences associées : TOM-9.1, TOM-9.2, TOM-9.4

Scénario 10 : Echec de l'authentification biométrique :

Objectif : Vérifier la gestion des erreurs lorsque l'authentification biométrique échoue.

Exigences associées : TOM-9.3

Scénario 11 : Désactivation de l'authentification biométrique :

Objectif : Vérifier que l'utilisateur peut désactiver la biométrie et revenir à une connexion classique.

Exigences associées : TOM-9.6

Scénario 12 : Gestion des données biométriques :

Objectif : Vérifier que les données biométriques sont stockées de manière sécurisée et ne peuvent pas être compromises.

Exigences associées : TOM-9.5

Scénario 13 : Sécurisation et gestion des justificatifs :

Objectif : Vérifier la sécurité et la gestion des justificatifs après leur téléversement.

Exigences associées : TOM-7.3, TOM-7.5, TOM-7.6, TOM-7.7

Scénario 14 : Configuration et sécurisation du double facteur :

Objectif : Vérifier si le double facteur peut être désactivé et comment se passe la récupération en cas de perte.

Exigences associées : TOM-8.2, TOM-8.5, TOM-8.6

Exigence 3 : Connexion sécurisée et intégration bancaire :

Description : Garantir la sécurité des échanges de données entre l'application et les services bancaires partenaires. S'assurer de la fiabilité de la connexion aux API externes (ex. : API BDF) ainsi que de la continuité de service en cas d'interruption.

Scénario 15: Connexion et sécurisation des échanges avec les banques :

Objectif : Vérifier que l'utilisateur peut se connecter à une banque compatible et que les échanges sont sécurisés.

Exigences associées : TOM-2.1, TOM-2.2, TOM-2.3. TOM-10.7. TOM-6.8

Scénario 16 : Blocage des appels HTTP et appels au HTTPS :

Objectif : Vérifier que les appels vers les API bancaires sont effectués exclusivement via HTTPS et que tout appel utilisant une URL HTTP est bloqué avec une erreur de sécurité.

Exigences associées : TOM-10.6

Scénario 17 : Gestion des interruptions et récupération des données :

Objectif : Vérifier la gestion des interruptions bancaires et la récupération des transactions.

Exigences associées : TOM-2.4, TOM-2.5, TOM-2.6

Scénario 18 : Connexion à l'API BDF et gestion des erreurs :

Objectif : Vérifier l'authentification et la gestion des erreurs lors de l'utilisation de l'API BDF.

Exigences associées : TOM-10.1, TOM-10.2, TOM-10.3

Scénario 19:Gestion des données bancaires et conservation des informations

Objectif : Vérifier la gestion des données bancaires stockées et affichées.

Exigences associées : TOM-10.4, TOM-10.5, TOM-11.1, TOM-11.2, TOM-11.3,

TOM-11.4, TOM-11.5

Scénario 20 : Téléchargement et gestion des relevés bancaires :

Objectif : Vérifier le bon fonctionnement du téléchargement des relevés et la traçabilité des actions.

Exigences associées : TOM-12.1, TOM-12.2, TOM-12.3, TOM-12.4, TOM-12.5,

TOM-12.6

Exigence 4 : Affichage et traitement des transactions :

Description : Offrir à l'utilisateur un affichage clair, fiable et actualisé de ses transactions. Gérer les erreurs liées aux appels API et assurer une bonne expérience utilisateur, y compris en cas de forte charge ou de regroupement d'informations.

Scénario 21 : Gestion de l'affichage des transactions :

Objectif : Vérifier que les transactions sont bien paginées, triées et filtrées.

Exigences associées : TOM-3.1, TOM-3.2, TOM-3.3

Scénario 22 : Lors d'une interconnexion bancaire réussie, le système récupère les informations bancaires de l'utilisateur et les enregistre dans les tables prévues.

Objectif : S'assurer que les informations bancaires de l'utilisateur sont bien récupérées et stockées correctement après une connexion bancaire réussie.

Exigences associées : TOM-6.9, TOM-10.7

Scénario 24 : Gestion des erreurs API :

Objectif : Vérifier le comportement du système en cas d'échec de l'API BDF.

Exigences associées : TOM-3.4

Scénario 25 : Gestion de l'historique des consultations :

Objectif : Vérifier que l'utilisateur peut consulter son historique.

Exigences associées : TOM-3.5

Scénario 26 : Rafraîchissement des transactions :

Objectif : Vérifier si les transactions sont mises à jour en temps réel ou à intervalle régulier.

Exigences associées : TOM-3.6

Scénario 27: Stockage et surcharge de l'API :

Objectif : Vérifier la gestion du stockage des transactions et la surcharge de l'API.

Exigences associées : TOM-13.1, TOM-13.2

Scénario 28 : Expérience utilisateur et affichage des transactions :

Objectif : Vérifier l'UX en cas de latence et la présence d'un indicateur de chargement.

Exigences associées : TOM-13.3, TOM-13.4

Scénario 29 : Regroupement et alerte des transactions :

Objectif : Vérifier le regroupement par type de dépense et la détection des transactions suspectes.

Exigences associées : TOM-13.5, TOM-13.6

Exigence 5 : Suivi, analyse et visualisation des données :

Description : Permettre à l'utilisateur de suivre sa consommation et d'analyser ses données via des représentations graphiques compréhensibles. Inclure des fonctionnalités d'export et assurer la compatibilité avec différents formats.

Scénario 30 : Analyse et affichage du suivi de consommation :

Objectif : Vérifier la gestion et l'affichage des dépenses.

Exigences associées : TOM-4.1, TOM-4.2, TOM-4.3, TOM-4.4, TOM-4.6

Scénario 31 : Export et compatibilité :

Objectif : Vérifier l'export des données et la gestion des devises.

Exigences associées : TOM-4.5, TOM-14.2

Scénario 32: Gestion des requêtes simultanées et affichage des graphiques:

Objectif : Vérifier la gestion des requêtes et les formats d’affichage.

Exigences associées : TOM-14.1, TOM-14.3, TOM-14.4, TOM-14.5

Exigence 6 : Recommandations et accompagnement utilisateur

Description : Fournir une assistance personnalisée à l’utilisateur via un conseiller virtuel. Adapter les recommandations en fonction des données collectées dans le respect de la confidentialité et des préférences de l’utilisateur.

Scénario 33 : Fonctionnement et personnalisation du conseiller virtuel :

Objectif : Vérifier la disponibilité et le fonctionnement du conseiller virtuel.

Exigences associées : TOM-5.1, TOM-5.2, TOM-5.3, TOM-5.4, TOM-5.5

Scénario 34 : Gestion des recommandations et des données collectées :

Objectif : S’assurer que les recommandations proposées par l’application sont utiles, logiques, modifiables et adaptées à la situation réelle de l’utilisateur.

Exigences associées : TOM-5.6, TOM-15.1, TOM-15.2, TOM-15.3, TOM-15.4, TOM-15.5

Méthodes de test adaptées :

On remarque que certains scénarios de test sont présents dans plusieurs catégories de tests. Cela s’explique par la nature du projet : plusieurs fonctionnalités sensibles ou critiques doivent être vérifiées sous différents angles. Elles doivent d’abord être validées sur le plan fonctionnel à travers des tests boîte noire, puis contrôlées à nouveau après chaque évolution grâce aux tests de non-régression. Enfin, certaines sont automatisées afin d’être rejouées régulièrement et de sécuriser les versions futures.

Cette organisation permet d’assurer à la fois la conformité fonctionnelle, la stabilité de l’application dans le temps et une meilleure efficacité des campagnes de test.

Ainsi, les scénarios les plus importants, notamment ceux liés à l’expérience utilisateur, à la sécurité ou à la gestion des données personnelles, sont volontairement couverts par plusieurs méthodes de test complémentaires.

Tests fonctionnels (tests boites noire) :

Les tests boîte noire sont des tests fonctionnels visant à vérifier que le système se comporte comme attendu sans avoir connaissance de son fonctionnement interne (code). On se concentre uniquement sur les entrées et les sorties. Cette approche se place du point de vue de l'utilisateur final.

Scénarios concernés :

Scénario 1 : Création d'un compte utilisateur :

Vérification fonctionnelle de la création avec différents cas d'entrées utilisateurs.

Scénario 2 : Évolution automatique de la date limite de naissance :

Validation du comportement dynamique de la limite d'âge sans accès au code.

Scénario 3 : modification des informations des comptes :

Vérification fonctionnelle des modifications décrites depuis l'interface utilisateur.

Scénario 4 : Suppression d'un compte :

Contrôle de la suppression d'un compte via les actions utilisateur.

Scénario 5 : Téléversement d'un justificatif :

Validation des contrôles de format et de taille à partir de l'interface.

Scénario 6 : Vérification du consentement et du stockage des données :

Validation de l'obtention du consentement RGPD à l'inscription.

Scénario 7 : Suppression automatique des données après inactivité :

Vérification du respect des durées de conservation sans accès à l'intérieur du traitement.

Scénario 8 : Authentification avec double facteur :

Validation du processus double facteur du point de vue utilisateur.

Scénario 9 : Activation de l'authentification biométrique :

Contrôle du bon fonctionnement de l'activation biométrique.

Scénario 10 : Échec de l'authentification biométrique :

Vérification de la gestion des erreurs d'authentification biométrique.

Scénario 11 : Désactivation de l'authentification biométrique :

Contrôle de la désactivation et retour au mot de passe.

Scénario 12 : Gestion des données biométriques :

Contrôle de la sécurisation des données biométriques.

Scénario 13 : Sécurisation et gestion des justificatifs :

Vérification du stockage sécurisé des fichiers téléversés.

Scénario 14 : Configuration et sécurisation du double facteur :

Validation de la configuration du double facteur et de sa récupération.

Scénario 15: Connexion et sécurisation des échanges avec les banques :

Vérification fonctionnelle des échanges API bancaires.

Scénario 17 : Gestion des interruptions et récupération des données :

Test fonctionnel des reconnections et sauvegardes de données.

Scénario 18 : Connexion à l'API BDF et gestion des erreurs :

Validation du comportement de l'authentification API

Scénario 19: Gestion des données bancaires et conservation des informations

Contrôle du stockage et de l'affichage des données bancaires.

Scénario 20 : Téléchargement et gestion des relevés bancaires :

Vérification du téléchargement et de l'historique.

Scénario 21 : Gestion de l’affichage des transactions :

Vérification de la pagination, tri, et recherche de transactions.

Scénario 24 : Gestion des erreurs API :

Test fonctionnel de l’affichage d’erreurs API.

Scénario 25 : Gestion de l’historique des consultations :

Contrôle du bon stockage et affichage des consultations.

Scénario 26 : Rafraîchissement des transactions :

Test du rafraîchissement manuel et automatique.

Scénario 28 : Expérience utilisateur et affichage des transactions :

Validation de l’affichage sous latence.

Scénario 29 : Regroupement et alerte des transactions :

Vérification du regroupement par type et détection d’anomalies.

Scénario 30 : Analyse et affichage du suivi de consommation :

Test de l’affichage et de la comparaison des dépenses.

Scénario 31 : Export et compatibilité :

Vérification des exports CSV/PDF et de la gestion des devises.

Scénario 32: Gestion des requêtes simultanées et affichage des graphiques:

Contrôle de l’affichage sous forte charge.

Scénario 33 : Fonctionnement et personnalisation du conseiller virtuel :

Test de la disponibilité et de la personnalisation des conseils.

Scénario 34 : Gestion des recommandations et des données collectées :

Validation des règles d’adaptation des recommandations.

Tests exploratoires :

Les tests exploratoires s’appuient sur l’expérience, l’observation et l’esprit d’analyse du testeur. Contrairement aux tests formalisés, ils ne suivent pas un scénario précis étape par étape. Le testeur explore l’application de manière plus libre afin de repérer d’éventuelles anomalies, des comportements inattendus ou des défauts qui n’auraient pas été identifiés dans les tests prévus. Ils viennent donc en complément des tests formels pour renforcer la qualité de l’application.

Scénarios concernés :

Scénario 2 : Évolution automatique de la date limite de naissance :

Exploration du calcul dynamique de l’âge.

Scénario 28 : Expérience utilisateur et affichage des transactions :

Détection d’anomalies d’affichage sous différentes latences.

Scénario 32 : Gestion des requêtes simultanées et affichage des graphiques :

Découverte d’éventuels problèmes d’affichage ou de lenteur en forte charge.

Scénario 33 : Fonctionnement et personnalisation du conseiller virtuel :

Vérification libre de la personnalisation et adaptabilité.

Tests de non-régression :

Les tests de non-régression ont pour objectif de vérifier que les fonctionnalités déjà opérationnelles continuent de fonctionner correctement après une modification de l'application. Ils permettent de détecter d'éventuels dysfonctionnements introduits involontairement à la suite d'une correction, d'une évolution ou d'une mise à jour du système.

Scénarios concernés :

Scénario 1 : Création d'un compte utilisateur :

S'assurer que la création reste fonctionnelle après évolutions.

Scénario 3 : Modification des informations d'un compte :

Contrôle de la persistance des modifications utilisateur.

Scénario 4 : Suppression d'un compte utilisateur :

S'assurer de la suppression correcte malgré les changements.

Scénario 7 : Suppression automatique des données après inactivité :

Validation de la récurrence du nettoyage automatique.

Scénario 8 : Authentification avec double facteur :

Maintenir le bon fonctionnement du double facteur.

Scénario 9 : Activation de l'authentification biométrique :

Contrôler la continuité de la fonctionnalité biométrique.

Scénario 10 : Échec de l'authentification biométrique :

Maintenir la gestion des erreurs biométriques.

Scénario 11 : Désactivation de l'authentification biométrique :

Contrôle de la désactivation sans régression.

Scénario 15 : Connexion et sécurisation des échanges avec les banques :

Continuité de la sécurisation des échanges API.

Scénario 17 : Gestion des interruptions et récupération des données :

S'assurer de la récupération des données en cas d'erreurs.

Scénario 18 : Connexion à l'API BDF et gestion des erreurs :

Maintenir les traitements d'erreurs de connexion.

Scénario 19 : Gestion des données bancaires et conservation des informations :

S'assurer de la conservation conforme des données bancaires.

Scénario 26 : Rafraîchissement des transactions :

Continuité du rafraîchissement automatique et manuel.

Scénario 27 : Stockage et surcharge de l'API :

Vérification que la surcharge est toujours gérée correctement.

Tests automatisés :

Les tests automatisés permettent d'exécuter de manière rapide et régulière des scénarios répétitifs, en particulier sur les parcours critiques. Ils contribuent à optimiser le temps de test tout en assurant la stabilité et la fiabilité du système.

Scénarios concernés :

Scénario 1 : Création d'un compte utilisateur :

Automatiser la création pour tester rapidement divers cas d'entrées.

Scénario 7 : Suppression automatique des données après inactivité :

Simuler et valider les suppressions sur le long terme.

Scénario 8 : Authentification avec double facteur :

Automatiser les vérifications de sécurité double facteur.

Scénario 9 : Activation de l'authentification biométrique :

Automatiser les activations et connexions biométriques.

Scénario 15 : Connexion et sécurisation des échanges avec les banques :

Tester automatiquement les différentes banques.

Scénario 17 : Gestion des interruptions et récupération des données :

Simuler des interruptions répétées.

Scénario 18 : Connexion à l'API BDF et gestion des erreurs :

Automatiser les cas d'erreurs et de timeout.

Scénario 20 : Téléchargement et gestion des relevés bancaires :

Répéter les téléchargements pour plusieurs configurations.

Scénario 23 : Gestion de l'affichage des transactions :

Automatiser les tests de tri, filtres et pagination.

Scénario 24 : Gestion des erreurs API :

Détecter rapidement les problèmes d'erreurs API.

Scénario 26 : Rafraîchissement des transactions :

Valider régulièrement la mise à jour automatique.

Scénario 28 : Expérience utilisateur et affichage des transactions :

Automatiser le test sous différentes conditions de réseau.

Scénario 32: Gestion des requêtes simultanées et affichage des graphiques :

Simuler des charges élevées pour détecter les ralentissements.

Ressources nécessaires :

Pour mettre en œuvre efficacement la stratégie de test définie, plusieurs types de ressources doivent être mobilisés. Celles-ci sont à la fois humaines, techniques et matérielles, afin d'assurer une couverture de test adaptée et de répondre aux exigences de qualité du projet.

Ressources humaines :

Testeurs fonctionnels : les tests manuels, notamment les tests en boîte noire, occupent une place importante dans la validation de l'application. Ils nécessitent des testeurs capables de bien comprendre le fonctionnement de l'outil afin de vérifier les parcours utilisateurs et les scénarios les plus critiques.

Testeurs techniques : la mise en œuvre des tests automatisés, en particulier pour la non-régression ou certains contrôles sur les API, nécessite au moins une ressource disposant de compétences en automatisation de tests, avec des outils adaptés à l'environnement du projet, tels que Cypress, Postman ou équivalent.

Compétences attendues :

- bonne connaissance des méthodes de tests fonctionnels, notamment les tests en boîte noire et les tests exploratoires ;
- bases en SQL pour vérifier certaines données enregistrées en base de données ;
- compréhension générale du fonctionnement des API REST afin de tester les échanges avec les services externes ;
- vigilance particulière concernant les exigences liées au RGPD, notamment pour la protection et la conservation des données personnelles ;
- compréhension fonctionnelle du domaine bancaire et assurantiel, en particulier sur les transactions, les relevés bancaires et la sécurisation des flux.

Outils de test :

Cahier de recette : le cahier de recette sera organisé par sprint, en cohérence avec le fonctionnement agile du projet. Cette organisation permettra de suivre plus facilement l'avancement des tests et d'assurer la traçabilité entre les exigences, les cas de test et les résultats obtenus.

Gestion des tests : un outil de gestion des tests sera nécessaire pour préparer, exécuter et suivre les campagnes de test, tout en garantissant une bonne traçabilité. Il pourra s'agir par exemple de TestRail, Xray, ou d'une organisation adaptée dans Jira ou Notion.

Automatisation : des outils d'automatisation comme Cypress pour les tests d'interface et Postman pour les tests API pourront être utilisés afin d'automatiser les scénarios critiques identifiés dans la stratégie de test.

Gestion des anomalies : l'utilisation d'un outil de suivi des anomalies, comme Jira, sera indispensable pour déclarer, suivre et prioriser efficacement les défauts détectés au cours des campagnes de test.

Données de test :

Comptes utilisateurs simulés : des comptes fictifs seront nécessaires pour tester les principales fonctionnalités liées aux utilisateurs, telles que l'inscription, la connexion, la suppression de compte et la mise à jour des informations personnelles.

Jeux de données bancaires : des données bancaires fictives, comme des transactions simulées, des historiques de relevés et des identifiants bancaires de test, seront utilisées pour valider l'interconnexion avec les services bancaires et la récupération des données.

Données sensibles protégées : afin de respecter les exigences du RGPD, seuls des jeux de données anonymisés ou fictifs devront être utilisés pendant les phases de test.

Environnements de test :

Environnement de préproduction : un environnement de test isolé, stable et proche de la production sera nécessaire pour exécuter les tests sans impacter les utilisateurs réels. Il devra reproduire au mieux les conditions de l'environnement final.

Accès aux environnements partenaires : l'accès aux environnements de test des partenaires bancaires ou des fournisseurs d'API sera également nécessaire, par exemple via des sandbox dédiées. Si ces environnements ne sont pas disponibles, des mocks ou des simulateurs devront être mis en place afin d'assurer la continuité des tests.

Disponibilité des environnements : la planification des tests devra tenir compte de la disponibilité de ces environnements afin de limiter les blocages, les conflits d'utilisation ou les interruptions.

Étapes clés de la stratégie :

La stratégie de test de l'application TOMSEN s'organise selon plusieurs étapes clés :

1. **Analyse de la documentation et revue des exigences :**
Identifier les fonctionnalités à tester, les règles de gestion et les points sensibles.
2. **Définition du périmètre de test :**
Déterminer les fonctionnalités et scénarios à couvrir.
3. **Choix des méthodes de test :**
Sélectionner les méthodes adaptées, notamment les tests fonctionnels, les tests d'intégration, les tests exploratoires et les tests de non-régression.
4. **Préparation des ressources de test :**
Préparer l'environnement, les données de test, les comptes utilisateurs et les outils de suivi.
5. **Définition des scénarios et cas de test :**
Construire les scénarios à partir de la revue des exigences et rédiger les cas de test associés.
6. **Planification des tests par sprint :**
Répartir les scénarios sur les 6 sprints en fonction des fonctionnalités développées.
7. **Exécution des tests et suivi des anomalies :**
Exécuter les tests, relever les anomalies et assurer leur suivi.
8. **Tests de non-régression et bilan :**
Vérifier le bon fonctionnement des fonctionnalités après correction et établir le bilan de la campagne de test.

Ces étapes générales structurent la démarche de test. Dans le cadre du projet TOMSEN, leur mise en œuvre s'inscrit dans une organisation agile en 6 sprints, permettant de planifier les activités de test de manière progressive tout au long du projet. Pour chaque sprint, une phase de test est prévue, comprenant la préparation des tests, leur exécution, le suivi des anomalies ainsi que les tests de non-régression.

Sprint 1 – Comptes et sécurité utilisateur :

Objectif de test :

Valider la gestion des comptes utilisateurs et les mécanismes de sécurité.

Scénarios de test associés :

- SC-1 : Création d'un compte utilisateur
- SC-2 : Évolution date de naissance
- SC-3 : Modification des informations
- SC-4 : Suppression de compte
- SC-5 : Téléversement justificatif
- SC-6 : Consentement et stockage des données
- SC-7 : Suppression des données (RGPD)
- SC-8 : Authentification double facteur
- SC-9 : Activation biométrie
- SC-10 : Échec biométrie
- SC-11 : Désactivation biométrie
- SC-12 : Gestion données biométriques
- SC-13 : Sécurisation justificatifs
- SC-14 : Configuration double facteur

Non-régression :

Vérification, après correction des anomalies, du bon fonctionnement des fonctionnalités déjà testées dans le sprint 1, notamment la création de compte, la gestion des informations utilisateur, le téléversement des justificatifs, l'authentification à double facteur et la biométrie.

Sprint 2 – Interconnexions bancaires + Dashboard :

Objectif

Tester la connexion aux banques et la récupération des données.

Scénarios :

- SC-15 : Connexion banques
- SC-16 : Sécurité HTTPS
- SC-17 : Gestion interruptions
- SC-18 : API BDF erreurs
- SC-19 : Gestion données bancaires
- SC-20 : Téléchargement relevés
- SC-21 : Blocage HTTP
- SC-22 : Récupération données bancaires

Non-régression

Authentification + sécurité API

Vérification, après correction des anomalies, du bon fonctionnement des mécanismes d'authentification et de la sécurisation des échanges avec les API, notamment la connexion aux services bancaires, l'utilisation obligatoire du protocole HTTPS, la gestion des erreurs d'appel et la protection des données échangées.

Sprint 3 – Consultation des données :

Objectif

Valider l'affichage et la manipulation des transactions.

Scénarios :

- SC-23 : Affichage transactions
- SC-24 : Gestion erreurs API
- SC-25 : Historique consultations
- SC-26 : Rafraîchissement données
- SC-27 : Surcharge API
- SC-28 : Expérience utilisateur

Sprint 4 – Suivi de consommation :

Objectif

Tester l'analyse financière et les graphiques.

Scénarios :

- SC-29 : Regroupement transactions
- SC-30 : Analyse consommation
- SC-31 : Export données
- SC-32 : Requêtes simultanées / graphiques

Sprint 5 – Conseiller virtuel :

Objectif

Valider les fonctionnalités d'IA et recommandations.

Scénarios :

- SC-33 : Conseiller virtuel
- SC-34 : Recommandations et données

Sprint 6 – Stabilisation globale :

Objectif

Valider l'application complète avant mise en production.

Activités :

- Tests de non-régression globaux
- Tests exploratoires
- Tests bout en bout (end-to-end)
- Vérification sécurité globale

- Vérification RGPD
- Validation finale métier

Préconisations :

Afin de réduire les risques identifiés lors de l'analyse des spécifications et de la revue d'exigences du projet TOMSEN, plusieurs préconisations peuvent être retenues :

- **Assurer la cohérence avec les objectifs du projet** en veillant à ce que les exigences, les user stories, les maquettes et les scénarios de test restent alignés avec les fonctionnalités prioritaires de l'application, notamment la sécurisation des comptes, l'interconnexion bancaire, la fiabilité des données affichées et l'accompagnement de l'utilisateur.
- **Planifier les tests dès le début de chaque sprint** afin d'anticiper la préparation des scénarios, l'exécution des tests, le suivi des anomalies et les tests de non-régression sur les fonctionnalités déjà validées.
- **Garantir la disponibilité des ressources nécessaires** à la campagne de test, en particulier un environnement de préproduction stable, des comptes fictifs, des jeux de données bancaires simulés, des justificatifs de test ainsi qu'un accès aux services partenaires ou à des simulateurs.
- **Préciser les exigences les plus sensibles du projet**, notamment celles liées à la création de compte, au téléversement des justificatifs, à l'authentification à double facteur, à la biométrie et aux échanges sécurisés avec les API bancaires et l'API Banque de France.
- **Prévoir des tests de non-régression systématiques** après chaque correction ou évolution afin de vérifier que les fonctionnalités déjà validées, comme la connexion, la récupération des données bancaires ou l'affichage des transactions, ne sont pas dégradées.
- **Envisager l'automatisation des scénarios les plus critiques et répétitifs**, notamment les parcours de connexion, les contrôles de formulaires et certains tests liés aux API, afin de fiabiliser les validations tout au long des sprints.